

# Autonomous Procurement Agent - Project Plan

## "Betsy" Implementation

---

### Case Summary

**Problem Context:** Operations managers like Jenny spend 30+ hours weekly on procurement firefighting--chasing invoices, preventing stockouts, managing supplier relationships reactively. This operational burden prevents strategic work and creates risk of production shutdowns, duplicate payments, and budget overruns.

**Solution Approach:** Build an autonomous procurement agent that monitors inventory levels, evaluates suppliers, generates purchase orders, reconciles invoices, and learns from outcomes--all without human intervention until approval stage.

**Target Outcome:** - 80% automation of procurement tasks - Prevent 2+ stockout scenarios - Catch 1+ invoice error - Maintain 95%+ human approval rate on autonomous decisions - Shift human focus from operational firefighting to strategic supplier relationships

**Success Criteria:** Agent operates autonomously (makes decisions while humans sleep), demonstrates multi-step reasoning (detect -> analyze -> decide -> act -> verify), maintains transparent decision trail for human review, learns from past decisions to improve supplier scoring, and implements safeguards against catastrophically bad decisions.

---

### Research Question

**Primary Question:** How do we architect autonomous procurement agents that make reliable, cost-effective purchasing decisions independently while maintaining transparency, learning from outcomes, and preventing catastrophic errors?

**Sub-Questions & Answers:**

- 1 What autonomous agent framework enables scheduled monitoring, multi-step workflows, and learning loops?
- 2 Framework comparison: LangGraph (known, good for workflows), AutoGen (multi-agent conversations), CrewAI (role-based), n8n (rapid prototyping)
- 3 Decision factors: scheduling capability, memory integration, learning system support
- 4 Recommendation direction: LangGraph (familiar, proven for complex workflows)
- 5 How should memory systems track supplier reliability over time?
- 6 Requirements: supplier history, pricing trends, delivery performance, past decision outcomes
- 7 Memory patterns: short-term (current transaction), mid-term (supplier scoring window), long-term (historical trends)
- 8 Storage considerations: Zep for session memory, structured DB for supplier profiles
- 9 Scoring evolution: how does one bad experience update reliability scores without eliminating good suppliers?
- 10 What decision logic prevents bad purchasing decisions?
- 11 Multi-factor evaluation: price vs delivery speed vs supplier reliability
- 12 EV-style calculations: purchase cost vs stockout prevention value

- 13 Threshold logic: when to choose expensive-but-fast vs cheap-but-slow
  - 14 Constraint checking: budget limits, approved supplier lists, maximum autonomous spend amounts
  - 15 How do autonomous agents handle edge cases?
  - 16 Supplier out of stock: failover to next supplier, escalate if all unavailable
  - 17 Delivery delays: proactive monitoring, early warning to human
  - 18 Price spikes: flag unusual pricing, require human approval above threshold
  - 19 Duplicate invoices: pattern matching against recent POs
  - 20 Budget constraints: hard limits vs soft warnings
  - 21 What safeguards prevent catastrophically bad decisions?
  - 22 Financial limits: maximum autonomous spend per transaction
  - 23 Human-in-loop: approval required for high-value or unusual decisions
  - 24 Audit trail: every decision logged with reasoning
  - 25 Rollback capability: how to reverse bad decisions
  - 26 Bias detection: fairness checks in supplier selection
  - 27 Privacy: sensitive business data handling protocols
  - 28 Should this use multi-agent or single-agent architecture?
  - 29 Multi-agent option: Monitor Agent, Evaluator Agent, Decision Agent, Reconciliation Agent, Learning Agent
  - 30 Single-agent option: One agent with different operational modes
  - 31 Trade-offs: modularity vs simplicity, independent optimization vs coordination overhead
  - 32 Recommendation: Start single-agent for MVP, evolve to multi-agent if complexity demands
- 

## Phase Timeline

### Phase 1: Architecture & Research (Week 1)

Objective: Define system architecture and select technical stack

Key Activities: - Research procurement workflows (interview someone in procurement/operations) - Compare agent frameworks (LangGraph, AutoGen, CrewAI, n8n) - Design memory architecture (supplier profiles, decision history) - Define decision logic framework (scoring algorithm, constraint checking) - Document ethical safeguards and risk mitigation strategies

Deliverables: - Framework selection decision (with justification) - Memory system design document - Decision logic specification - Risk assessment and safeguard plan - Decision log entry: Architecture choices

Success Criteria: - Clear rationale for framework choice - Memory system handles all required data types - Decision logic addresses edge cases - Safeguards cover identified risks

---

### Phase 2: Core Autonomous Loop - Manual Simulation (Week 2)

Objective: Build and test basic detect -> decide -> act workflow with manual data

Key Activities: - Create mock inventory data with stockout scenarios - Implement threshold detection logic - Build supplier evaluation system (hardcoded options initially) - Generate purchase order format - Implement human approval workflow - Test with 3-5 simulated scenarios

Deliverables: - Working prototype (manual triggers) - Mock data generators - PO generation capability - Test scenario results - Decision log entry: Initial implementation approach

Success Criteria: - Agent detects stockout conditions correctly - Supplier evaluation logic runs without errors - PO format matches business requirements - Human can approve/reject with clear reasoning visible

---

### **Phase 3: Memory & Learning Integration (Week 3)**

Objective: Add memory system and enable learning from decisions

Key Activities: - Implement supplier history tracking - Build supplier scoring system (performance-based) - Add pricing trend analysis - Create learning loop (decision -> outcome -> score update) - Test that second decision improves based on first outcome - Implement delivery tracking and delay detection

Deliverables: - Memory system integration - Supplier scoring algorithm - Learning feedback loop - Comparative test results (1st vs 2nd decision quality) - Decision log entry: Memory architecture implementation

Success Criteria: - Supplier scores update after transactions - Agent makes better decisions with more history - Memory persists across sessions - Pricing trends influence future decisions

---

### **Phase 4: Edge Cases & Robustness (Week 4)**

Objective: Test failure modes and implement recovery strategies

Key Activities: - Test supplier out-of-stock scenario - Test delivery delay handling - Test price spike detection - Implement duplicate invoice detection - Test budget constraint violations - Create escalation workflows for unhandled cases

Deliverables: - Edge case test suite - Recovery strategy implementations - Escalation workflow documentation - Invoice reconciliation system - Decision log entry: Edge case handling strategies

Success Criteria: - Agent handles all identified edge cases gracefully - No silent failures (all errors logged/escalated) - Duplicate invoice detection catches test cases - Budget constraints prevent overspend

---

### **Phase 5: Safeguards, Ethics & Demo Prep (Week 5)**

Objective: Implement safety measures and prepare final demonstration

Key Activities: - Implement financial limits (max autonomous spend) - Build audit trail system (decision reasoning logs) - Conduct bias analysis (supplier selection fairness) - Create human override mechanisms - Test with realistic end-to-end scenarios - Document ethical considerations - Prepare demonstration materials

Deliverables: - Complete safeguard implementation - Audit trail documentation - Ethics section for decision log - Final demonstration scenarios - Complete decision log with all phases documented - Presentation materials

Success Criteria: - Agent meets all success metrics (2+ stockout prevention, 1+ invoice error caught, 95%+ approval) - Safeguards prevent catastrophic decisions - Audit trail enables human understanding of all decisions - Ethics documentation addresses transparency, accountability, fairness, safety, privacy - Demo shows both happy path and edge case

## Upcoming Development Plan

### Immediate Next Steps (This Week)

- 1 Domain Research - Interview or research procurement workflows, pain points, decision criteria
- 2 Framework Selection - Evaluate frameworks against requirements, make selection decision
- 3 Architecture Design - Single vs multi-agent decision, memory system structure
- 4 Mock Data Creation - Design inventory data structure, supplier profiles, realistic scenarios

### Technical Decisions Required

- ☐ Framework: LangGraph vs AutoGen vs CrewAI vs n8n vs custom
- ☐ Memory: Zep vs custom DB vs hybrid approach
- ☐ Agent pattern: Single-agent with modes vs multi-agent system
- ☐ Scheduling: How does agent run autonomously (cron, event-driven, continuous loop)?
- ☐ Integration: Mock APIs vs real API stubs vs full integration

### Build Strategy

Incremental approach: - Week 2: Manual triggering, hardcoded data -> validate decision logic  
- Week 3: Add memory persistence -> validate learning works - Week 4: Add error handling  
-> validate robustness - Week 5: Add safety constraints -> validate safeguards work

Testing philosophy: - Don't just show happy path - Demonstrate edge cases, errors, unexpected situations - Show agent learning over time (decision N+1 better than decision N)

### Documentation Strategy

Decision log entries at each phase: - Research findings and framework rationale - Technical architecture choices - Failed approaches and pivots - Edge case discoveries - Ethical safeguard implementation - Final reflections on autonomy vs safety trade-offs

---

## Stakeholder Analysis

### Primary Stakeholder: Operations Manager (Jenny)

Needs: - Reduce time spent on procurement firefighting (30+ hours/week -> strategic work) - Prevent production line shutdowns (stockout prevention) - Catch errors humans miss (duplicate invoices, pricing discrepancies) - Maintain control (approval workflow for autonomous decisions) - Understand agent reasoning (transparent decision trail)

Concerns: - Will agent make costly mistakes? - Can I override when needed? - How do I know why agent chose supplier X over Y? - What happens when agent encounters something unexpected?

Success from Jenny's perspective: - "I work ON procurement now, not IN procurement" - Sunday evenings are free - Proactive notifications instead of Monday morning chaos -

## Secondary Stakeholder: Finance Team

Needs: - Budget compliance (no unauthorized overruns) - Accurate invoice reconciliation - Duplicate payment prevention - Audit trail for all purchases

Concerns: - Can agent be trusted with company funds? - What's the maximum damage if something goes wrong? - How do we audit autonomous decisions?

Success from Finance perspective: - Reduced duplicate payments - Better budget visibility - Clean audit trail

---

## Secondary Stakeholder: Production/Assembly Line

Needs: - No stockouts (continuous material availability) - Fast response to urgent needs - Minimal production disruptions

Concerns: - Will agent prioritize cost over speed when speed matters? - What if agent chooses slow supplier for critical part?

Success from Production perspective: - Zero stockout incidents - Materials arrive before they're needed - Proactive rather than reactive procurement

---

## Tertiary Stakeholder: Suppliers

Needs: - Fair evaluation and selection - Consistent ordering patterns - Timely payments

Concerns: - Will agent show bias toward certain suppliers? - Will agent exploit automated negotiation? - How transparent is the selection process?

Success from Supplier perspective: - Fair chance to compete for business - No unexplained selection patterns - Predictable ordering behavior

---

## Ethical Stakeholder: Future Users & Society

Needs: - Trustworthy autonomous systems - Understandable AI decision-making - Safe deployment patterns

Concerns: - Are autonomous agents making high-stakes decisions without sufficient oversight? - Who's responsible when autonomous agents fail? - Are we building systems that perpetuate biases?

Success from Ethical perspective: - Transparent decision-making - Clear accountability mechanisms - Demonstrated safety constraints - Bias detection and mitigation

---

## Constraints

### Technical Constraints

- Time: 5-week school project timeline (iterative implementation required)
- Complexity: Must be demonstrable/testable within academic context (likely simulated integrations)
- Infrastructure: No access to real procurement systems (requires mock data and APIs)
- Scale: Small manufacturing company context (10-50 SKUs, 5-10 suppliers realistic scope)

- Integration: Email, inventory systems, supplier APIs must be mocked or stubbed

## Functional Constraints

- Autonomy boundaries: Agent cannot operate completely unsupervised (human approval workflow required)
- Financial limits: Maximum autonomous spend amount must be defined
- Supplier limitations: Pre-approved supplier list (agent doesn't find new suppliers autonomously)
- Scope boundaries: Focus on material procurement, not services/contracts/capital equipment

## Performance Constraints

- Success metrics: Must prevent 2+ stockouts, catch 1+ invoice error, achieve 95%+ approval rate
- Response time: Stockout detection must occur with enough lead time for delivery
- Learning speed: Agent must show improvement within demonstration timeframe (5-10 decisions)

## Safety Constraints

- No catastrophic decisions: Safeguards must prevent scenarios like ordering 1000x quantity, selecting unapproved suppliers, exceeding budget by orders of magnitude
- Reversibility: Bad decisions must be identifiable and correctable
- Escalation: Edge cases beyond agent capability must escalate to human
- Transparency: All decisions must be explainable to stakeholders

## Ethical Constraints

- Fairness: No bias toward/against suppliers based on non-business factors
- Privacy: Business data must be handled appropriately (in real deployment)
- Accountability: Clear audit trail for every autonomous decision
- Human oversight: Humans remain accountable for agent actions

## Educational Constraints

- Documentation required: Decision log must track research, choices, failures, pivots
  - Demonstration required: Must show working system with realistic scenarios
  - Learning objective: Understanding autonomous agent architecture, not just implementation
  - Reflection required: Ethics section on safeguards and what could go wrong
- 

# Risk Assessment & Mitigation

## High-Risk Scenarios

- 1 Agent makes \$50,000 mistake
- 2 Mitigation: Maximum autonomous spend limit (\$5k-10k), human approval for high-value
- 3 Agent selects wrong supplier causing production shutdown
- 4 Mitigation: Delivery time evaluation, reliability scoring, buffer time in stockout detection
- 5 Agent exhibits supplier bias

- 6 Mitigation: Regular bias audits, diverse supplier scoring factors, fairness checks
- 7 Agent misses duplicate invoice
- 8 Mitigation: Multiple duplicate detection strategies, invoice pattern matching
- 9 Edge case causes infinite loop or crash
- 10 Mitigation: Timeout mechanisms, error handling, escalation workflows

## Medium-Risk Scenarios

- 1 Agent makes sub-optimal decisions (penny-wise, pound-foolish)
- 2 Mitigation: Learning from outcomes, EV-style calculations including shutdown costs
- 3 Supplier out of stock not detected
- 4 Mitigation: Availability checking before PO generation, failover logic
- 5 Budget drift over time
- 6 Mitigation: Running budget tracking, alerts at 80% threshold

---

Project Start Date: 2026-05-27 Target Completion: 5 weeks from start (2026-07-01) Next Review: End of Phase 1 (Architecture decisions)

---

This project plan will evolve as decisions are made and challenges are discovered. All significant changes will be documented in the decision log.